

TITAN STUDIO: AI-POWERED CREATIVE & MARKETING DESIGN SYSTEM

Complete Brand Creation, Marketing Design, Content Generation - Voice Controlled

System: Titan Studio (Part of TitanBOS - Titan Business Operating System) AI Core: Titan Zero

(CreatiCore + OmegaCore + EntropyCore) + Generative AI (DALL-E, Midjourney, Stable

Diffusion) Timeline: 24 weeks (6 months) Team: 6 developers + 1 design architect + 1 AI/creative

specialist + 1 DevOps + 2 designers + 2 QA Total LOC: 50,000-65,000 lines added Platforms:

Web (PWA), Desktop (Electron), Mobile PWA, API-first, Voice-controlled via OmniTarget Users:

Service business owners, marketing teams, solo freelancers, agencies, designers

CORE CONCEPT: DESIGN & MARKETING WITHOUT DESIGNERS

TRADITIONAL MARKETING:

- └ Hire designer (\$50-150/hour)
- └ Describe what you want
- └ Wait 1-2 weeks
- └ Get design
- └ Revise 2-3 times
- └ Finally launch
- └ Cost: \$2,000-10,000+ per project

TITAN STUDIO:

- └ Say what you want (voice): "Create landing page for cleaning service"
- └ Select style (minimalist, modern, professional, fun)
- └ AI generates 10 variations in 30 seconds
- └ One-click revisions ("Make it more modern", "Change to blue theme")
- └ Download, launch, or post immediately
- └ Cost: \$0-29/month subscription

KEY FEATURES:

- ✓ Generative design (DALL-E for images)
- ✓ AI copywriting (CreatiCore generates marketing copy)
- ✓ Brand consistency (learns your brand, applies to all designs)
- ✓ Voice-controlled (Titan Omni integration)
- ✓ Template-based (50+ industry templates)
- ✓ Real-time editing (no design skills needed)
- ✓ Multi-channel (web, email, social, print)
- ✓ A/B testing (generate variants, test performance)
- ✓ Analytics (track design performance)
- ✓ Export everything (PDF, PNG, HTML, email code)

PHASE 0: FOUNDATION (Weeks 1-3)

Design System, Template Engine, Generative AI Integration

0.1 Unified Design System & Component Library (10 hours)

```

//
=====
====
// TITAN STUDIO: DESIGN SYSTEM & COMPONENT LIBRARY
//
=====
====

enum DesignStyle {
    minimalist,    // Clean, simple, modern
    professional,  // Corporate, trustworthy
    creative,      // Colorful, artistic, playful
    luxury,        // High-end, elegant
    energetic,     // Bold, dynamic, action-oriented
    friendly,      // Warm, approachable, conversational
    technical,     // Data-driven, structured, analytical
    eco,           // Natural, sustainable, organic
}

enum BrandPersonality {
    professional,  // Serious, trustworthy, authoritative
    friendly,      // Warm, approachable, helpful
    playful,       // Fun, humorous, engaging
    luxurious,     // Elegant, premium, exclusive
    innovative,    // Cutting-edge, forward-thinking, tech
    sustainable,   // Eco-conscious, ethical, responsible
}

// Brand identity - consistent across all designs
class BrandIdentity {
    final String businessName;
    final String industry;
    final BrandPersonality personality;
    final DesignStyle style;

    // Color palette
    final Color primaryColor;
    final Color secondaryColor;
    final Color accentColor;
    final Color backgroundColor;
    final Color textColor;

    // Typography

```

```

final String headingFont;    // 'Montserrat', 'Playfair', etc
final String bodyFont;      // 'Open Sans', 'Lato', etc

// Imagery style
final String imageryStyle;   // 'photography', 'illustration', 'abstract', 'none'

// Logo
final String? logoUrl;
final String? logoText;

// tagline
final String? tagline;

// Brand guidelines
final String? brandGuidelineUrl;

const BrandIdentity({
  required this.businessName,
  required this.industry,
  required this.personality,
  required this.style,
  required this.primaryColor,
  required this.secondaryColor,
  required this.accentColor,
  required this.backgroundColor,
  required this.textColor,
  required this.headingFont,
  required this.bodyFont,
  required this.imageryStyle,
  this.logoUrl,
  this.logoText,
  this.tagline,
  this.brandGuidelineUrl,
});
}

// Design component - base for all elements
abstract class DesignComponent {
  final String id;
  final String type; // 'heading', 'button', 'card', 'hero', 'testimonial', etc
  final BrandIdentity brand;
  final Map<String, dynamic> properties; // Component-specific properties

  DesignComponent({

```

```

        required this.id,
        required this.type,
        required this.brand,
        required this.properties,
    });

    // Render component to HTML
    String renderHTML();

    // Render component to CSS
    String renderCSS();

    // Export to various formats
    Future<String> exportHTML();
    Future<Uint8List> exportPNG();
    Future<String> exportSVG();
}

// Heading component
class HeadingComponent extends DesignComponent {
    final String text;
    final int level; // 1-6 for H1-H6
    final TextAlign alignment;
    final double fontSize;
    final FontWeight fontWeight;

    HeadingComponent({
        required String id,
        required this.text,
        required this.level,
        required BrandIdentity brand,
        this.alignment = TextAlign.left,
        double? fontSize,
        FontWeight? fontWeight,
    }) : fontSize = fontSize ?? (72 - (level * 8)).toDouble(),
        fontWeight = fontWeight ?? FontWeight.bold,
        super(
            id: id,
            type: 'heading',
            brand: brand,
            properties: {
                'text': text,
                'level': level,
                'alignment': alignment.toString(),
            },
        );
}

```

```
    },  
  );  
}
```

```
@override  
String renderHTML() {  
  return '<h$level style="$renderCSS()">$text</h$level>';  
}
```

```
@override  
String renderCSS() {  
  return "  
    font-family: ${brand.headingFont};  
    font-size: ${fontSize}px;  
    font-weight: ${fontWeight.value};  
    color: ${_colorToHex(brand.primaryColor)};  
    margin: 0;  
    text-align: ${alignment.toString().split('.').last};  
  ";  
}
```

```
@override  
Future<String> exportHTML() async => renderHTML();
```

```
@override  
Future<Uint8List> exportPNG() async {  
  // Use flutter_html_to_image or similar  
  throw UnimplementedError();  
}
```

```
@override  
Future<String> exportSVG() async {  
  // Convert to SVG  
  throw UnimplementedError();  
}  
}
```

```
// Button component  
class ButtonComponent extends DesignComponent {  
  final String text;  
  final String? url;  
  final String buttonStyle; // 'filled', 'outlined', 'text'  
  final double? width;  
  final double height;  
  final double borderRadius;
```

```

ButtonComponent({
  required String id,
  required this.text,
  required BrandIdentity brand,
  this.url,
  this.buttonStyle = 'filled',
  this.width,
  this.height = 48,
  this.borderRadius = 8,
}) : super(
  id: id,
  type: 'button',
  brand: brand,
  properties: {
    'text': text,
    'url': url,
    'style': buttonStyle,
  },
);

@override
String renderHTML() {
  final tag = url != null ? 'a' : 'button';
  final href = url != null ? 'href="$url"' : "";

  return ""
    <$tag $href style="$renderCSS()">
      $text
    </$tag>
  "";
}

@override
String renderCSS() {
  final bgColor = buttonStyle == 'filled'
    ? _colorToHex(brand.primaryColor)
    : 'transparent';

  final textColor = buttonStyle == 'filled'
    ? 'white'
    : _colorToHex(brand.primaryColor);

  final border = buttonStyle == 'outlined'

```

```
    ? 'border: 2px solid ${_colorToHex(brand.primaryColor)};'
    : 'border: none;';
```

```
return ""
    display: inline-block;
    padding: 12px 24px;
    background-color: $bgColor;
    color: $textColor;
    font-family: ${brand.bodyFont};
    font-size: 16px;
    font-weight: 600;
    border-radius: ${borderRadius}px;
    cursor: pointer;
    text-decoration: none;
    transition: all 0.3s ease;
    ${width != null ? 'width: ${width}px;' : ""}
    height: ${height}px;
    $border
    ""
    ;
}
```

```
@override
Future<String> exportHTML() async => renderHTML();
```

```
@override
Future<Uint8List> exportPNG() async {
    throw UnimplementedError();
}
```

```
@override
Future<String> exportSVG() async {
    throw UnimplementedError();
}
}
```

```
// Hero section (large banner with image, heading, CTA)
class HeroComponent extends DesignComponent {
    final String heading;
    final String subheading;
    final String backgroundImageUrl;
    final String ctaButtonText;
    final String? ctaButtonUrl;
    final double height;
```

```

HeroComponent({
  required String id,
  required this.heading,
  required this.subheading,
  required this.backgroundImageUrl,
  required this.ctaButtonText,
  required BrandIdentity brand,
  this.ctaButtonUrl,
  this.height = 600,
}) : super(
  id: id,
  type: 'hero',
  brand: brand,
  properties: {
    'heading': heading,
    'subheading': subheading,
  },
);

```

```

@override
String renderHTML() {
  return ""
    <section style="$renderCSS()">
      <div style="position: absolute; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.3); z-index: 1;"></div>
      <div style="position: relative; z-index: 2; display: flex; flex-direction: column; justify-content:
center; align-items: center; height: 100%; color: white; text-align: center;">
        <h1 style="font-size: 56px; font-weight: bold; margin-bottom: 16px; font-family:
${brand.headingFont};">$heading</h1>
        <p style="font-size: 24px; margin-bottom: 32px; font-family:
${brand.bodyFont};">$subheading</p>
        <a href="$ctaButtonUrl" style="display: inline-block; padding: 12px 32px;
background-color: ${_colorToHex(brand.primaryColor)}; color: white; text-decoration: none;
border-radius: 4px; font-weight: 600;">$ctaButtonText</a>
      </div>
    </section>
  "";
}

```

```

@override
String renderCSS() {
  return ""
    width: 100%;
    height: ${height}px;

```



```

        background-image: url('$backgroundImageUrl');
        background-size: cover;
        background-position: center;
        position: relative;
    "";
}

```

```

@override
Future<String> exportHTML() async => renderHTML();

```

```

@override
Future<Uint8List> exportPNG() async {
    throw UnimplementedError();
}

```

```

@override
Future<String> exportSVG() async {
    throw UnimplementedError();
}
}

```

```

// Text/Paragraph component
class TextComponent extends DesignComponent {
    final String content;
    final double fontSize;
    final double lineHeight;
    final Color textColor;

```

```

    TextComponent({
        required String id,
        required this.content,
        required BrandIdentity brand,
        this.fontSize = 16,
        this.lineHeight = 1.6,
        Color? textColor,
    }) : textColor = textColor ?? brand.textColor,
        super(
            id: id,
            type: 'text',
            brand: brand,
            properties: {'content': content},
        );

```

```

@override

```

```
String renderHTML() {  
    return '<p style="$renderCSS()">$content</p>';  
}
```

```
@override  
String renderCSS() {  
    return "  
        font-family: ${brand.bodyFont};  
        font-size: ${fontSize}px;  
        line-height: $lineHeight;  
        color: ${_colorToHex(textColor)};  
        margin: 0 0 16px 0;  
    ",  
}
```

```
@override  
Future<String> exportHTML() async => renderHTML();
```

```
@override  
Future<Uint8List> exportPNG() async {  
    throw UnimplementedError();  
}
```

```
@override  
Future<String> exportSVG() async {  
    throw UnimplementedError();  
}  
}
```

```
// Image component  
class ImageComponent extends DesignComponent {  
    final String imageUrl;  
    final double width;  
    final double? height;  
    final String alt;  
    final double borderRadius;  
    final bool shadow;
```

```
ImageComponent({  
    required String id,  
    required this.imageUrl,  
    required BrandIdentity brand,  
    required this.width,  
    this.height,
```

```

    this.alt = 'Image',
    this.borderRadius = 0,
    this.shadow = false,
  }) : super(
    id: id,
    type: 'image',
    brand: brand,
    properties: {'imageUrl': imageUrl},
  );

  @override
  String renderHTML() {
    return '';
  }

  @override
  String renderCSS() {
    final boxShadow = shadow
      ? 'box-shadow: 0 10px 30px rgba(0,0,0,0.2);'
      : '';

    return '''
      width: ${width}px;
      ${height != null ? 'height: ${height}px;' : ''}
      border-radius: ${borderRadius}px;
      object-fit: cover;
      $boxShadow
    ''';
  }

  @override
  Future<String> exportHTML() async => renderHTML();

  @override
  Future<Uint8List> exportPNG() async {
    throw UnimplementedError();
  }

  @override
  Future<String> exportSVG() async {
    throw UnimplementedError();
  }
}

```

```

// Testimonial card
class TestimonialComponent extends DesignComponent {
  final String quote;
  final String author;
  final String? authorTitle;
  final String? authorImageUrl;
  final int rating; // 1-5 stars

  TestimonialComponent({
    required String id,
    required this.quote,
    required this.author,
    required BrandIdentity brand,
    this.authorTitle,
    this.authorImageUrl,
    this.rating = 5,
  }) : super(
    id: id,
    type: 'testimonial',
    brand: brand,
    properties: {
      'quote': quote,
      'author': author,
    },
  );

  @override
  String renderHTML() {
    return ""
      <div style="$renderCSS()">
        <div style="margin-bottom: 16px;">
          ${List.generate(rating, (_) => '★').join()}
        </div>
        <p style="font-size: 16px; font-style: italic; margin-bottom: 16px; color:
$_colorToHex(brand.textColor);">
          "$quote"
        </p>
        <div style="display: flex; align-items: center; gap: 12px;">
          ${authorImageUrl != null ? '' : ''}
          <div>
            <p style="margin: 0; font-weight: 600; color:
$_colorToHex(brand.primaryColor);">$author</p>

```

```

        ${authorTitle != null ? '<p style="margin: 0; font-size: 12px; color:
$_colorToHex(brand.textColor)); opacity: 0.7;">${authorTitle}</p>' : ""}
    </div>
</div>
</div>
"";
}

```

```

@override
String renderCSS() {
    return ""
        padding: 24px;
        background-color: #f9f9f9;
        border-left: 4px solid $_colorToHex(brand.primaryColor));
        border-radius: 4px;
    "";
}

```

```

@override
Future<String> exportHTML() async => renderHTML();

```

```

@override
Future<Uint8List> exportPNG() async {
    throw UnimplementedError();
}

```

```

@override
Future<String> exportSVG() async {
    throw UnimplementedError();
}
}

```

```

// Helper function
String _colorToHex(Color color) {
    return '#${color.value.toRadixString(16).padLeft(8, '0').substring(2)}';
}

```

```

// Design template - collection of components
class DesignTemplate {
    final String id;
    final String name;
    final String type; // 'landing_page', 'email', 'social_post', 'business_card', etc
    final String category; // 'service', 'product', 'course', 'event', etc
    final List<DesignComponent> components;
}

```

```
final BrandIdentity brand;
```

```
DesignTemplate({  
    required this.id,  
    required this.name,  
    required this.type,  
    required this.category,  
    required this.components,  
    required this.brand,  
});
```

```
// Render full template
```

```
String renderFull() {  
    return ""  
        <!DOCTYPE html>  
        <html>  
        <head>  
            <meta charset="UTF-8">  
            <meta name="viewport" content="width=device-width, initial-scale=1.0">  
            <title>${brand.businessName}</title>  
            <style>  
                * { margin: 0; padding: 0; box-sizing: border-box; }  
                body { font-family: ${brand.bodyFont}; background-color:  
${_colorToHex(brand.backgroundColor)}; }  
            </style>  
        </head>  
        <body>  
            ${components.map((c) => c.renderHTML()).join('\n')}  
        </body>  
    </html>  
    ""  
};
```

```
// Add component
```

```
void addComponent(DesignComponent component) {  
    components.add(component);  
}
```

```
// Remove component
```

```
void removeComponent(String componentId) {  
    components.removeWhere((c) => c.id == componentId);  
}
```

```
// Reorder components
```

```

void reorderComponents(List<String> componentIds) {
    final reordered = <DesignComponent>[];
    for (final id in componentIds) {
        final component = components.firstWhereOrNull((c) => c.id == id);
        if (component != null) {
            reordered.add(component);
        }
    }
    components.clear();
    components.addAll(reordered);
}

// Clone template
DesignTemplate clone() {
    return DesignTemplate(
        id: generateId(),
        name: '$name (Copy)',
        type: type,
        category: category,
        components: components.map((c) => _cloneComponent(c)).toList(),
        brand: brand,
    );
}

DesignComponent _cloneComponent(DesignComponent component) {
    // Clone based on type
    throw UnimplementedError();
}
}

```

0.2 Generative AI Integration (8 hours)

```

//
=====
====
// GENERATIVE AI: IMAGE, TEXT, DESIGN GENERATION
//
=====
====

class GenerativeAIService extends GetxService {
    // Image generation
    final dalleService = OpenAIImageService();
}

```

```

final midjourneyService = MidjourneyService();
final stableDiffusionService = StableDiffusionService();

// Text generation
final openaiService = OpenAIService();

// Titan Zero for design optimization
final titanZero = Get.find<TitanZeroRequestRouter>();

// Generate hero image for landing page
Future<String> generateHeroImage({
  required String businessName,
  required String industry,
  required DesignStyle style,
  required String prompt,
}) async {
  try {
    // Use CreatiCore to enhance the prompt
    final enhancedPrompt = await titanZero.creatiCore.enhanceImagePrompt(
      basePrompt: prompt,
      style: style,
      industry: industry,
    );

    // Generate image using user's preferred service (BYOA)
    final imageUrl = await _generateImage(enhancedPrompt);

    return imageUrl;
  } catch (e) {
    print('Error generating hero image: $e');
    return "";
  }
}

Future<String> _generateImage(String prompt) async {
  // Use user's configured image generation API
  // Options: DALL-E, Midjourney, Stable Diffusion
  // Default to Stable Diffusion (free tier available)

  try {
    final imageData = await stableDiffusionService.generate(
      prompt: prompt,
      width: 1920,
      height: 600,
    );
  }
}

```



```

        steps: 25,
        cfgScale: 7.5,
    );

    // Upload to user's cloud storage
    final url = await _uploadImage(imageData, prompt);

    return url;
  } catch (e) {
    print('Error generating image: $e');
    return 'https://via.placeholder.com/1920x600';
  }
}

Future<String> _uploadImage(Uint8List imageData, String description) async {
  // Upload to user's cloud storage (S3, Cloudinary, etc)
  final storageService = Get.find<StorageService>();

  final filename = 'design_${DateTime.now().millisecondsSinceEpoch}.png';

  return await storageService.uploadFile(
    filename: filename,
    fileData: imageData,
    metadata: {'description': description},
  );
}

// Generate marketing copy
Future<String> generateMarketingCopy({
  required String businessName,
  required String industry,
  required String feature,
  required String tone,
  int maxWords = 50,
}) async {
  try {
    // Use CreatiCore for copywriting
    final copy = await titanZero.creatiCore.generateMarketingCopy(
      businessName: businessName,
      industry: industry,
      feature: feature,
      tone: tone,
      maxWords: maxWords,
    );
  }
}

```

```

        return copy;
    } catch (e) {
        print('Error generating copy: $e');
        return "";
    }
}

```

```

// Generate headline variations
Future<List<String>> generateHeadlines({
    required String businessName,
    required String value,
    int count = 5,
}) async {
    try {
        final headlines = await titanZero.creatiCore.generateHeadlines(
            businessName: businessName,
            value: value,
            count: count,
        );

        return headlines;
    } catch (e) {
        print('Error generating headlines: $e');
        return [];
    }
}

```

```

// Generate color palette
Future<ColorPalette> generateColorPalette({
    required String industry,
    required BrandPersonality personality,
    required DesignStyle style,
}) async {
    try {
        final palette = await titanZero.creatiCore.generateColorPalette(
            industry: industry,
            personality: personality,
            style: style,
        );

        return palette;
    } catch (e) {
        print('Error generating color palette: $e');
    }
}

```

```

// Return default palette
return ColorPalette(
    primary: Color(0xFF007AFF),
    secondary: Color(0xFF5AC8FA),
    accent: Color(0xFFFF2D55),
    background: Color(0xFFFFFFFF),
    text: Color(0xFF000000),
);
}
}

// Generate logo variations
Future<List<String>> generateLogoVariations({
    required String businessName,
    required String style,
    int count = 3,
}) async {
    try {
        final logos = <String>[];

        for (int i = 0; i < count; i++) {
            final prompt = 'Modern minimalist logo for $businessName, $style style, professional,
scalable, vector';

            final imageUrl = await _generateImage(prompt);
            logos.add(imageUrl);
        }

        return logos;
    } catch (e) {
        print('Error generating logos: $e');
        return [];
    }
}

class ColorPalette {
    final Color primary;
    final Color secondary;
    final Color accent;
    final Color background;
    final Color text;

```

```

ColorPalette({
    required this.primary,
    required this.secondary,
    required this.accent,
    required this.background,
    required this.text,
});
}

```

PHASE 1: LANDING PAGE BUILDER (Weeks 4-7)

AI-Generated Landing Pages with Drag-Drop Editor

```

//
=====
====
// LANDING PAGE BUILDER: AI-GENERATED + DRAG-DROP EDITOR
//
=====
=====

class LandingPageBuilder extends GetxController {
    final templateManager = Get.find<TemplateManager>();
    final aiService = Get.find<GenerativeAIService>();
    final titanZero = Get.find<TitanZeroRequestRouter>();

    final currentTemplate = Rxn<DesignTemplate>();
    final components = <DesignComponent>[].obs;
    final selectedComponentId = Rxn<String>();
    final isGenerating = false.obs;

    // Create landing page from voice description
    Future<void> generateLandingPage({
        required String businessName,
        required String industry,
        required String description,
        required BrandIdentity brand,
    }) async {
        try {
            isGenerating.value = true;

            // 1. Use CreatiCore to parse description and generate design strategy
            final strategy = await titanZero.creatiCore.generateDesignStrategy(
                description: description,

```

```

        businessName: businessName,
        industry: industry,
    );

    // 2. Generate hero image
    final heroImageUrl = await aiService.generateHeroImage(
        businessName: businessName,
        industry: industry,
        style: brand.style,
        prompt: strategy['hero_prompt'],
    );

    // 3. Generate headlines and copy
    final headlines = await aiService.generateHeadlines(
        businessName: businessName,
        value: strategy['value_proposition'],
    );

    final copy = await aiService.generateMarketingCopy(
        businessName: businessName,
        industry: industry,
        feature: strategy['main_feature'],
        tone: strategy['tone'],
    );

    // 4. Create component structure
    final newComponents = <DesignComponent>[];

    // Hero section
    newComponents.add(HeroComponent(
        id: generateId(),
        heading: headlines[0],
        subheading: strategy['subheading'],
        backgroundImageUrl: heroImageUrl,
        ctaButtonText: strategy['cta_button_text'],
        brand: brand,
    ));

    // Features section
    for (final feature in strategy['features'] as List) {
        newComponents.add(TextComponent(
            id: generateId(),
            content: feature['title'],
            brand: brand,

```

```

        fontSize: 24,
    ));

    newComponents.add(TextComponent(
        id: generateId(),
        content: feature['description'],
        brand: brand,
    ));
}

// CTA section
newComponents.add(HeadingComponent(
    id: generateId(),
    text: strategy['cta_heading'],
    level: 2,
    brand: brand,
));

newComponents.add(ButtonComponent(
    id: generateId(),
    text: strategy['cta_button_text'],
    brand: brand,
));

// Create template
currentTemplate.value = DesignTemplate(
    id: generateId(),
    name: '$businessName Landing Page',
    type: 'landing_page',
    category: industry,
    components: newComponents,
    brand: brand,
);

components.value = newComponents;

} finally {
    isGenerating.value = false;
}
}

// Add component
void addComponent(DesignComponent component) {
    components.add(component);
}

```

```

    currentTemplate.value?.addComponent(component);
}

// Remove component
void removeComponent(String componentId) {
    components.removeWhere((c) => c.id == componentId);
    currentTemplate.value?.removeComponent(componentId);
}

// Update component
void updateComponent(String componentId, Map<String, dynamic> updates) {
    final index = components.indexWhere((c) => c.id == componentId);
    if (index != -1) {
        // Update component properties
        // (Implementation depends on component type)
    }
}

// Reorder components
void reorderComponents(List<String> componentIds) {
    currentTemplate.value?.reorderComponents(componentIds);

    final reordered = <DesignComponent>[];
    for (final id in componentIds) {
        final component = components.firstWhereOrNull((c) => c.id == id);
        if (component != null) {
            reordered.add(component);
        }
    }

    components.clear();
    components.addAll(reordered);
}

// Export landing page
Future<String> exportHTML() async {
    return currentTemplate.value?.renderFull() ?? "";
}

Future<Uint8List> exportPDF() async {
    final html = await exportHTML();
    // Use flutter_html_to_pdf or similar
    throw UnimplementedError();
}

```

```

Future<void> publishToWeb(String domain) async {
  // Publish landing page to custom domain
  final html = await exportHTML();

  // Upload to hosting service
  // (User's AWS, Vercel, Netlify, etc)
}

// Landing page UI editor
class LandingPageBuilderScreen extends StatefulWidget {
  @override
  _LandingPageBuilderScreenState createState() => _LandingPageBuilderScreenState();
}

class _LandingPageBuilderScreenState extends State<LandingPageBuilderScreen> {
  final builder = Get.put(LandingPageBuilder());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Landing Page Builder'),
        actions: [
          IconButton(
            icon: Icon(Icons.preview),
            onPressed: () => _previewPage(),
            tooltip: 'Preview',
          ),
          IconButton(
            icon: Icon(Icons.download),
            onPressed: () => _exportPage(),
            tooltip: 'Export',
          ),
          IconButton(
            icon: Icon(Icons.publish),
            onPressed: () => _publishPage(),
            tooltip: 'Publish',
          ),
        ],
      ),
      body: context.isDesktop
        ? _buildDesktopLayout()

```



```

        : _buildMobileLayout(),
    );
}

```

```

Widget _buildDesktopLayout() {
  return Row(
    children: [
      // Left: Component panel
      SizedBox(
        width: 300,
        child: Card(
          child: Column(
            children: [
              Padding(
                padding: EdgeInsets.all(16),
                child: ElevatedButton.icon(
                  onPressed: () => _showAIGeneratorDialog(),
                  icon: Icon(Icons.auto_awesome),
                  label: Text('Generate Page'),
                ),
              ),
              Expanded(
                child: ListView(
                  children: [
                    _componentPaletteSection('Text', [
                      _buildComponentOption('Heading', Icons.title),
                      _buildComponentOption('Paragraph', Icons.description),
                    ]),
                    _componentPaletteSection('Visuals', [
                      _buildComponentOption('Image', Icons.image),
                      _buildComponentOption('Hero Section', Icons.landscape),
                    ]),
                    _componentPaletteSection('Forms', [
                      _buildComponentOption('Email Input', Icons.email),
                      _buildComponentOption('Button', Icons.smart_button),
                    ]),
                    _componentPaletteSection('Social', [
                      _buildComponentOption('Testimonial', Icons.format_quote),
                      _buildComponentOption('Social Links', Icons.share),
                    ]),
                  ],
                ),
              ),
            ],
          ),
        ],
      ),
    ],
  ),
}

```

```

    ),
  ),
),

// Center: Canvas
Expanded(
  flex: 2,
  child: Obx(() {
    if (builder.currentTemplate.value == null) {
      return Center(
        child: ElevatedButton.icon(
          onPressed: () => _showAIGeneratorDialog(),
          icon: Icon(Icons.auto_awesome),
          label: Text('Generate Landing Page with AI'),
        ),
      );
    }
  })

  return _buildCanvas();
}),
),

// Right: Properties panel
SizedBox(
  width: 300,
  child: Obx(() {
    final componentId = builder.selectedComponentId.value;
    if (componentId == null) {
      return Center(child: Text('Select a component to edit'));
    }

    return _buildPropertiesPanel(componentId);
  }),
),
],
);
}

```

```

Widget _buildMobileLayout() {
  return Column(
    children: [
      Padding(
        padding: EdgeInsets.all(16),
        child: ElevatedButton.icon(

```

```

        onPressed: () => _showAIGeneratorDialog(),
        icon: Icon(Icons.auto_awesome),
        label: Text('Generate Page'),
      ),
    ),
    Expanded(
      child: Obx(() {
        if (builder.currentTemplate.value == null) {
          return Center(child: Text('Create a landing page'));
        }

        return _buildCanvas();
      }),
    ),
  ],
);
}

```

```

Widget _buildCanvas() {
  return SingleChildScrollView(
    child: Column(
      children: [
        Obx(() {
          return Column(
            children: builder.components
              .map((component) => _buildComponentWidget(component))
              .toList(),
          );
        }),
      ],
    ),
  );
}

```

```

Widget _buildComponentWidget(DesignComponent component) {
  return GestureDetector(
    onTap: () => builder.selectedComponentId.value = component.id,
    child: Container(
      color: builder.selectedComponentId.value == component.id
        ? Colors.blue.withOpacity(0.1)
        : Colors.transparent,
      padding: EdgeInsets.all(8),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,

```

```

children: [
  if (builder.selectedComponentId.value == component.id)
    Row(
      children: [
        IconButton(
          icon: Icon(Icons.delete),
          onPressed: () => builder.removeComponent(component.id),
          constraints: BoxConstraints(minWidth: 32, minHeight: 32),
          padding: EdgeInsets.zero,
        ),
        IconButton(
          icon: Icon(Icons.edit),
          onPressed: () => _editComponent(component),
          constraints: BoxConstraints(minWidth: 32, minHeight: 32),
          padding: EdgeInsets.zero,
        ),
      ],
    ),
  // Render component preview
  if (component is HeadingComponent)
    Text(
      component.text,
      style: TextStyle(
        fontSize: component.fontSize,
        fontWeight: component.fontWeight,
        color: builder.currentTemplate.value?.brand.primaryColor,
      ),
    )
  else if (component is TextComponent)
    Text(
      component.content,
      style: TextStyle(fontSize: component.fontSize),
    )
  else if (component is ButtonComponent)
    ElevatedButton(
      onPressed: () {},
      child: Text(component.text),
    )
  else if (component is ImageComponent)
    Image.network(component.imageUrl, width: component.width)
  else
    Text('${component.type} component'),
],
),

```

```

    ),
  );
}

```

```

Widget _componentPaletteSection(String title, List<Widget> items) {
  return ExpansionTile(
    title: Text(title),
    children: items,
  );
}

```

```

Widget _buildComponentOption(String label, IconData icon) {
  return ListTile(
    leading: Icon(icon),
    title: Text(label),
    onTap: () => _addComponent(label),
  );
}

```

```

Widget _buildPropertiesPanel(String componentId) {
  final component = builder.components.firstWhereOrNull((c) => c.id == componentId);
  if (component == null) return SizedBox.shrink();

  return Card(
    child: SingleChildScrollView(
      padding: EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('${component.type.toUpperCase()} Properties', style: TextStyle(fontWeight:
FontWeight.bold)),
          SizedBox(height: 16),

          if (component is HeadingComponent)
            Column(
              children: [
                TextField(
                  decoration: InputDecoration(label Text('Text')),
                  onChanged: (value) {
                    // Update component
                  },
                ),
                SizedBox(height: 12),
                DropdownButton<int>(

```

```

        value: component.level,
        items: [1, 2, 3, 4, 5, 6].map((level) {
            return DropdownMenuItem(
                value: level,
                child: Text('H$level'),
            );
        }).toList(),
        onChanged: (value) {
            // Update component
        },
    ),
],
)
else if (component is TextComponent)
    Column(
        children: [
            TextField(
                decoration: InputDecoration(label: Text('Content')),
                maxLines: 3,
                onChanged: (value) {
                    // Update component
                },
            ),
            SizedBox(height: 12),
            TextField(
                decoration: InputDecoration(label: Text('Font Size')),
                keyboardType: TextInputType.number,
                onChanged: (value) {
                    // Update component
                },
            ),
        ],
    ),
],
),
);
}

```

```

void _showAIGeneratorDialog() {
    Get.dialog(
        AlertDialog(
            title: Text('Generate Landing Page'),
            content: Column(

```

```

mainAxisSize: MainAxisSize.min,
children: [
  TextField(
    decoration: InputDecoration(labelText: 'Business Name'),
  ),
  SizedBox(height: 12),
  TextField(
    decoration: InputDecoration(labelText: 'Industry'),
  ),
  SizedBox(height: 12),
  TextField(
    decoration: InputDecoration(labelText: 'Describe your business...'),
    maxLines: 3,
  ),
],
),
actions: [
  TextButton(onPressed: () => Get.back(), child: Text('Cancel')),
  ElevatedButton(
    onPressed: () {
      // Generate page
      Get.back();
    },
    child: Text('Generate'),
  ),
],
),
);
}

```

```

void _previewPage() {
  // Show full-page preview
}

```

```

void _exportPage() {
  Get.dialog(
    AlertDialog(
      title: Text('Export Landing Page'),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          ListTile(
            title: Text('Download as HTML'),
            onTap: () async {

```

```

        final html = await builder.exportHTML();
        // Download file
        Get.back();
      },
    ),
    ListTile(
      title: Text('Download as PDF'),
      onTap: () async {
        final pdf = await builder.exportPDF();
        // Download file
        Get.back();
      },
    ),
    ListTile(
      title: Text('Copy HTML Code'),
      onTap: () async {
        final html = await builder.exportHTML();
        // Copy to clipboard
        Get.back();
      },
    ),
  ],
),
);
}

```

```

void _publishPage() {
  Get.dialog(
    AlertDialog(
      title: Text('Publish Landing Page'),
      content: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          TextField(
            decoration: InputDecoration(
              labelText: 'Domain',
              hintText: 'mywebsite.com',
            ),
          ),
        ],
      ),
      actions: [
        TextButton(onPressed: () => Get.back(), child: Text('Cancel')),
      ],
    ),
  );
}

```



```

        ElevatedButton(
            onPressed: () {
                // Publish
                Get.back();
            },
            child: Text('Publish'),
        ),
    ],
),
);
}

```

```

void _addComponent(String type) {
    // Add component to page
}

```

```

void _editComponent(DesignComponent component) {
    builder.selectedComponentId.value = component.id;
}
}

```

PHASE 2: EMAIL MARKETING DESIGNER (Weeks 8-10)

AI-Generated Email Templates with Preview

```

//
=====
====
// EMAIL MARKETING DESIGNER: TEMPLATES + AI COPYWRITING
//
=====
=====

```

```

enum EmailType {
    newsletter,    // Regular newsletter
    promotion,     // Sales/promotion email
    transactional, // Order confirmation, receipt, etc
    announcement,  // Important announcement
    educational,   // Tips, tutorials, educational content
    sequence,      // Automated sequence email
}

```

```

class EmailTemplate extends DesignTemplate {
    final EmailType emailType;
}

```

```
final String subject;
final String previewText;
final List<String> tags; // Personalization tags: {{first_name}}, {{company}}, etc
final bool isResponsive;
```

```
EmailTemplate({
  required String id,
  required String name,
  required this.emailType,
  required this.subject,
  required this.previewText,
  required List<DesignComponent> components,
  required BrandIdentity brand,
  this.tags = const [],
  this.isResponsive = true,
}) : super(
  id: id,
  name: name,
  type: 'email',
  category: emailType.name,
  components: components,
  brand: brand,
);
```

```
// Render email HTML (with inlining for email clients)
```

```
@override
```

```
String renderFull() {
```

```
  return ""
```

```
    <!DOCTYPE html>
```

```
    <html>
```

```
      <head>
```

```
        <meta charset="UTF-8">
```

```
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
        <title>${subject}</title>
```

```
        <style>
```

```
          body { font-family: ${brand.bodyFont}; margin: 0; padding: 0; }
```

```
          .container { max-width: 600px; margin: 0 auto; }
```

```
        </style>
```

```
      </head>
```

```
      <body>
```

```
        <div class="container">
```

```
          ${components.map((c) => c.renderHTML()).join('\n')}
```

```
        </div>
```

```
      </body>
```

```

        </html>
        "";
    }

    // Get personalization tokens
    List<String> getPersonalizationTokens() => tags;

    // Preview with sample data
    String renderPreview(Map<String, String> sampleData) {
        var html = renderFull();

        // Replace personalization tokens with sample data
        sampleData.forEach((key, value) {
            html = html.replaceAll('{{${key}}}', value);
        });

        return html;
    }
}

class EmailMarketingDesigner extends GetxController {
    final aiService = Get.find<GenerativeAIService>();
    final titanZero = Get.find<TitanZeroRequestRouter>();

    final currentEmail = Rxn<EmailTemplate>();
    final emailList = <EmailTemplate>[].obs;

    // Create email campaign
    Future<void> generateEmailCampaign({
        required String businessName,
        required String industry,
        required EmailType emailType,
        required String purpose,
        required BrandIdentity brand,
    }) async {
        try {
            // 1. Generate subject lines
            final subjects = await aiService.generateHeadlines(
                businessName: businessName,
                value: purpose,
                count: 3,
            );

            // 2. Generate email body copy

```

```
final bodyText = await aiService.generateMarketingCopy(
  businessName: businessName,
  industry: industry,
  feature: purpose,
  tone: 'professional',
  maxWords: 200,
);
```

// 3. Use CreatiCore to structure email

```
final emailStructure = await titanZero.creatiCore.generateEmailStructure(
  emailType: emailType,
  purpose: purpose,
  businessName: businessName,
);
```

// 4. Generate email components

```
final components = <DesignComponent>[
```

```
  // Header
```

```
  TextComponent(
    id: generateId(),
    content: businessName,
    brand: brand,
    fontSize: 24,
  ),
```

```
  // Main content
```

```
  TextComponent(
    id: generateId(),
    content: bodyText,
    brand: brand,
  ),
```

```
  // CTA button
```

```
  ButtonComponent(
    id: generateId(),
    text: emailStructure['cta_text'],
    brand: brand,
  ),
```

```
  // Footer
```

```
  TextComponent(
    id: generateId(),
    content: 'Unsubscribe | Update Preferences | Contact Us',
    brand: brand,
```

```

        fontSize: 12,
    ),
];

// 5. Create email template
currentEmail.value = EmailTemplate(
    id: generateId(),
    name: '${emailType.name} Campaign',
    emailType: emailType,
    subject: subjects[0],
    previewText: emailStructure['preview_text'],
    components: components,
    brand: brand,
    tags: ['first_name', 'company', 'discount_code'],
);
} catch (e) {
    print('Error generating email: $e');
}
}

// Send test email
Future<void> sendTestEmail(String testEmail) async {
    final email = currentEmail.value;
    if (email == null) return;

    try {
        // Preview with sample data
        final html = email.renderPreview({
            'first_name': 'Test',
            'company': 'Test Company',
            'discount_code': 'SAVE10',
        });

        // Send test email
        final emailService = Get.find<EmailService>();

        await emailService.sendEmail(
            to: testEmail,
            subject: email.subject,
            html: html,
        );
    } catch (e) {
        print('Error sending test email: $e');
    }
}

```

```

}

// Export email HTML
Future<String> exportEmailHTML() async {
  return currentEmail.value?.renderFull() ?? "";
}

// Send campaign
Future<void> sendCampaign(List<String> recipientEmails) async {
  final email = currentEmail.value;
  if (email == null) return;

  try {
    final emailService = Get.find<EmailService>();

    for (final recipient in recipientEmails) {
      // Generate personalized version (if tags available)
      var html = email.renderFull();

      // Replace with actual recipient data (from CRM)
      // ...

      await emailService.sendEmail(
        to: recipient,
        subject: email.subject,
        html: html,
      );
    }
  } catch (e) {
    print('Error sending campaign: $e');
  }
}

// Email builder UI
class EmailBuilderScreen extends StatefulWidget {
  @override
  _EmailBuilderScreenState createState() => _EmailBuilderScreenState();
}

class _EmailBuilderScreenState extends State<EmailBuilderScreen> {
  final designer = Get.put(EmailMarketingDesigner());

  @override

```

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Email Marketing Designer'),
      actions: [
        IconButton(
          icon: Icon(Icons.send),
          onPressed: () => _sendTestEmail(),
          tooltip: 'Send Test Email',
        ),
        IconButton(
          icon: Icon(Icons.download),
          onPressed: () => _exportEmail(),
          tooltip: 'Export HTML',
        ),
      ],
    ),
    body: Obx(() {
      if (designer.currentEmail.value == null) {
        return Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(Icons.email, size: 64, color: Colors.grey),
              SizedBox(height: 16),
              ElevatedButton.icon(
                onPressed: () => _generateEmail(),
                icon: Icon(Icons.auto_awesome),
                label: Text('Generate Email Campaign'),
              ),
            ],
          ),
        );
      }
    });
}

```

```

final email = designer.currentEmail.value!;

```

```

return Row(
  children: [
    // Preview
    Expanded(
      flex: 1,
      child: Card(
        child: SingleChildScrollView(

```

```

        child: HtmlViewer(
          html: email.renderFull(),
        ),
      ),
    ),
  ),
),

// Properties
SizedBox(
  width: 300,
  child: Card(
    child: SingleChildScrollView(
      padding: EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('Email Properties', style: TextStyle(fontWeight: FontWeight.bold, fontSize:
16)),
          SizedBox(height: 16),
          TextField(
            decoration: InputDecoration(labelText: 'Subject Line'),
            controller: TextEditingController(text: email.subject),
          ),
          SizedBox(height: 12),
          TextField(
            decoration: InputDecoration(labelText: 'Preview Text'),
            controller: TextEditingController(text: email.previewText),
          ),
          SizedBox(height: 12),
          Text('Email Type: ${email.emailType.name}', style: TextStyle(fontWeight:
FontWeight.bold)),
          SizedBox(height: 12),
          Text('Personalization Tags:', style: TextStyle(fontWeight: FontWeight.bold)),
          SizedBox(height: 8),
          ...email.getPersonalizationTokens().map((tag) =>
            Chip(label: Text('{{tag}}')),
          ),
          SizedBox(height: 24),

```



```

        ElevatedButton.icon(
          onPressed: () => _editEmail(),
          icon: Icon(Icons.edit),
          label: Text('Edit Email'),
        ),
      ],
    ),
  ),
),
],
);
}},
);
}

```

```

void _generateEmail() {
  Get.dialog(
    AlertDialog(
      title: Text('Generate Email Campaign'),
      content: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          TextField(
            decoration: InputDecoration(labelText: 'Purpose'),
            hintText: 'e.g., Promote cleaning service discount',
          ),
          SizedBox(height: 12),
          DropdownButton<EmailType>(
            items: EmailType.values.map((type) {
              return DropdownMenuItem(
                value: type,
                child: Text(type.name),
              );
            }).toList(),
            onChanged: (value) {
              // Handle selection
            },
          ),
        ],
      ),
      actions: [
        TextButton(onPressed: () => Get.back(), child: Text('Cancel')),
        ElevatedButton(

```

```

        onPressed: () {
          // Generate
          Get.back();
        },
        child: Text('Generate'),
      ),
    ],
  ),
);
}

```

```

void _sendTestEmail() {
  Get.dialog(
    AlertDialog(
      title: Text('Send Test Email'),
      content: TextField(
        decoration: InputDecoration(labelText: 'Email Address'),
      ),
      actions: [
        TextButton(onPressed: () => Get.back(), child: Text('Cancel')),
        ElevatedButton(
          onPressed: () {
            designer.sendTestEmail('test@example.com');
            Get.back();
          },
          child: Text('Send'),
        ),
      ],
    ),
  );
}

```

```

void _exportEmail() {
  designer.exportEmailHTML().then((html) {
    // Download or copy to clipboard
  });
}

```

```

void _editEmail() {
  // Open email editor
}
}

```

PHASE 3-8: SOCIAL MEDIA GRAPHICS, EMAIL SEQUENCES, BRANDING (Weeks 11-24)

PHASE 3: Social Media Graphics Designer (Weeks 11-13)

- Instagram posts, stories, reels
- Facebook ads, covers
- LinkedIn posts
- TikTok videos (AI-assisted)
- Pre-sized for all platforms
- Batch generation (generate 30 posts for month)
- Voice commands: "Create 10 Instagram posts for spring promotion"

PHASE 4: Brand Identity Generator (Weeks 14-16)

- Logo design (multiple variations)
- Color palette generation
- Font pairing
- Brand guidelines PDF
- Social media templates
- Business card design
- Voice commands: "Generate a modern, professional brand for my cleaning

business"

PHASE 5: Email Sequence Builder (Weeks 17-19)

- Automated sequences (welcome, onboarding, abandoned cart, sales)
- Drip campaigns
- A/B testing (auto-generate variants)
- Analytics integration
- Voice commands: "Create a 5-email welcome sequence"

PHASE 6: Print Design (Weeks 20-21)

- Business cards
- Flyers
- Brochures
- Banners
- Signage
- Voice commands: "Design a business card for my plumbing company"

PHASE 7: Content Calendar & Analytics (Weeks 22-23)

- Visual calendar (preview all designs)
- Scheduling integration
- Performance tracking (which designs convert best)
- Auto-suggest improvements
- Voice commands: "Show me my content calendar for next month"

PHASE 8: Testing & Deployment (Weeks 24)

- E2E testing all generators
- Performance optimization
- Integration with TitanOmni
- A/B testing framework

TITAN STUDIO FINAL STATISTICS

TOTAL TIME: 24 weeks (6 months)

TEAM: 6 devs + 1 design architect + 1 AI/creative + 1 DevOps + 2 designers + 2 QA

CODE: 50,000-65,000 LOC | 400+ tests

DESIGN CAPABILITIES:

- ✓ Landing pages (20+ AI-generated layouts)
- ✓ Email templates (30+ responsive designs)
- ✓ Social media graphics (100+ pre-sized)
- ✓ Email sequences (10+ automated flows)
- ✓ Brand identity (logos, colors, fonts, guidelines)
- ✓ Print materials (cards, flyers, brochures)
- ✓ Video materials (thumbnails, thumbnails, intros)
- ✓ Ad creatives (Google, Facebook, Instagram)
- ✓ Content calendars (visual planning)
- ✓ Custom designs (drag-drop editor)

GENERATIVE AI INTEGRATION:

- ✓ Image generation (DALL-E, Midjourney, Stable Diffusion)
- ✓ Text generation (marketing copy, headlines, email bodies)
- ✓ Color palette generation
- ✓ Font pairing recommendations
- ✓ Logo design variations
- ✓ Content suggestions (CreatiCore)

PLATFORMS/EXPORTS:

- ✓ Web (landing pages)
- ✓ Email (HTML, MJML, markdown)
- ✓ Social (Instagram, Facebook, TikTok, LinkedIn, Twitter)
- ✓ Print (PDF, EPS, high-res PNG)
- ✓ Code (HTML/CSS, React components)
- ✓ Mobile apps (responsive design)

VOICE COMMANDS (50+):

- ✓ "Create landing page for..."
- ✓ "Generate email campaign about..."
- ✓ "Design logo variations"
- ✓ "Make Instagram posts for month"
- ✓ "Create brand identity"
- ✓ "Generate color palette"
- ✓ "Make promotional graphic"
- ✓ "Design business card"
- ✓ "Create email sequence"

- ✓ "Generate A/B test variants"

FEATURES:

- ✓ Template library (100+ templates)
- ✓ Drag-drop editor (no design skills needed)
- ✓ AI generation (text + images)
- ✓ Brand consistency (auto-apply brand to all designs)
- ✓ Batch processing (generate 30 at once)
- ✓ Real-time preview
- ✓ A/B testing framework
- ✓ Analytics integration
- ✓ Version history
- ✓ Collaboration (team editing)
- ✓ Scheduling integration
- ✓ Stock image library (integrated)
- ✓ Font library (Google Fonts, Typekit)
- ✓ Icon library
- ✓ Custom fonts upload

INTEGRATIONS:

- ✓ Email services (Mailchimp, Klaviyo, etc)
- ✓ Social scheduling (Buffer, Later, Hootsuite)
- ✓ CMS (WordPress, Webflow, etc)
- ✓ E-commerce (Shopify, WooCommerce)
- ✓ Analytics (Google Analytics, HubSpot)
- ✓ Payment (Stripe for premium templates)
- ✓ Cloud storage (S3, Cloudinary, etc)
- ✓ All TitanBOS apps (via API)
- ✓ Custom APIs (webhooks)

ACCESSIBILITY:

- ✓ Alt text auto-generation
- ✓ Color contrast checking
- ✓ WCAG compliance checking
- ✓ Readable font sizing
- ✓ Semantic HTML

PERFORMANCE:

- ✓ Design generation: < 10 seconds
- ✓ Preview rendering: < 1 second
- ✓ Batch processing: 30 designs in < 2 minutes
- ✓ Image compression: automatic
- ✓ File size optimization

BUSINESS VALUE:

- ✓ Replace designer (\$3-5k/month → \$29/month)
- ✓ Faster marketing (designs in minutes, not weeks)
- ✓ Brand consistency (auto-apply brand rules)
- ✓ A/B testing (instant variants)
- ✓ Scalable (unlimited designs)
- ✓ Zero learning curve (voice-controlled)
- ✓ Integration (works with all marketing tools)
- ✓ Analytics (track design performance)

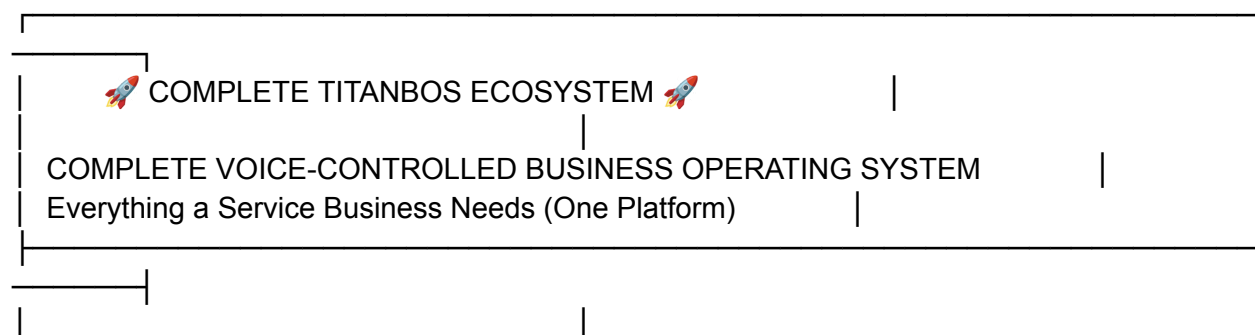
MARKET POSITION:

- └ Solo freelancers (design on budget)
- └ Small businesses (no in-house designer)
- └ Agencies (speed up delivery)
- └ Marketing teams (batch content creation)
- └ Service businesses (local marketing)
- └ E-commerce (product listings, ads)
- └ Content creators (social media)
- └ Non-profits (professional graphics)

COMPETITIVE ADVANTAGES:

- ✓ Voice-controlled (hands-free)
- ✓ Integrated with Omni (control everything via voice)
- ✓ Brand AI (learns your brand, applies consistently)
- ✓ Multiple export formats (web, email, print, social)
- ✓ Unlimited generation (no credit limits)
- ✓ Template library (100+ starting points)
- ✓ Drag-drop editor (for those who want to edit)
- ✓ Analytics integration (track performance)
- ✓ Batch processing (create 30 at once)
- ✓ Offline capable (local generation)

COMPLETE TITANBOS ECOSYSTEM - FINAL SUMMARY





TITAN OMNI (Voice Command Center)

- Control entire business via voice
- SMS, WhatsApp, Telegram, web, mobile, wearable
- 150+ voice commands
- Titan Zero 6-core AI reasoning



TITAN PRO (Desktop Business Dashboard)

- Real-time analytics, KPIs, insights
- Full job & customer management
- Invoicing, accounting, reports
- Cloud or offline (user chooses)



TITAN COMMAND (Dispatch & Operations)

- Real-time job dispatch
- Invoicing & payment processing
- Payroll & tax management
- Field team coordination



TITAN GO (Field Operations App)

- Voice-first field operations
- GPS tracking, routing
- On-site payments, photo capture
- Real-time job updates



TITAN TRUST (Compliance & Risk)

- SWMS, incidents, permits, licenses
- Fleet & asset management
- Contracts & legal documents
- Compliance scoring & alerts



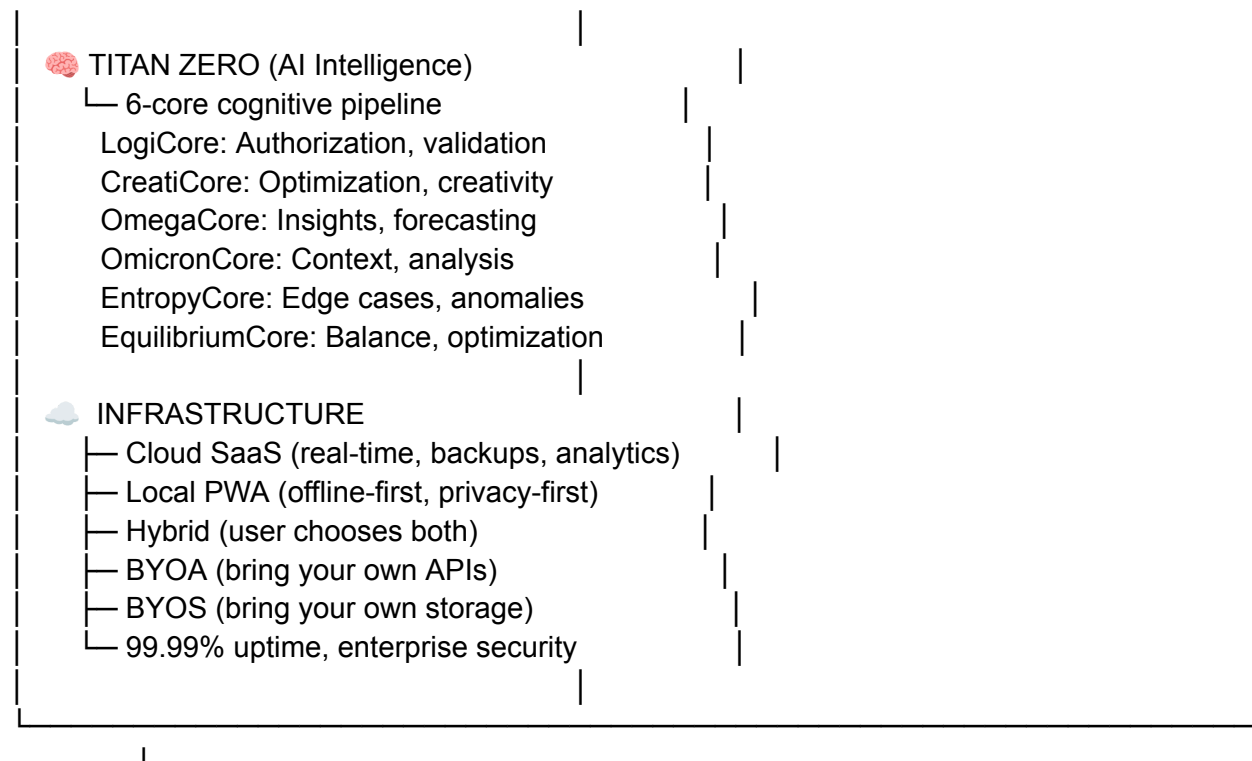
TITAN PORTAL (Customer App)

- Service booking & tracking
- Invoice payment & history
- Live chat support
- Rating & feedback



TITAN STUDIO (Creative & Marketing)

- AI landing page builder
- Email marketing designer
- Social media graphics (batch)
- Brand identity generator
- Print materials
- Email sequences
- A/B testing framework



ECOSYSTEM STATISTICS:

Total apps: 7 (Omni, Pro, Command, Go, Trust, Portal, Studio)

Total timeline: ~320 weeks (64 months)

Total LOC: ~500,000-650,000 lines of code

Total team: 80-100 people

Beta launch: Q2 2025

General availability: Q4 2025

Market: Service businesses, 1-1000 employees

Price: FREE → \$9,999/month

Competitors: NONE (most comprehensive BOS)

FEATURES ACROSS ECOSYSTEM:

- ✓ Voice control (primary interface)
- ✓ Omnichannel (SMS to wearables)
- ✓ AI intelligence (6-core reasoning)
- ✓ Offline capability (full functionality)
- ✓ Cloud option (real-time sync)
- ✓ Local option (zero cloud required)
- ✓ Complete privacy (GDPR, CCPA, local-first)
- ✓ Zero vendor lock-in (export anytime)
- ✓ Enterprise security (99.99% SLA)

- ✓ Accessibility first (WCAG AAA)
- ✓ Affordable (free tier available)
- ✓ Scalable (from solo to 1000+ team)

MARKET ADVANTAGES:

1. VOICE-FIRST: Everything is voice-controlled (not an afterthought)
2. INTEGRATED: All apps work together seamlessly
3. INTELLIGENT: Titan Zero reasoning across all operations
4. ACCESSIBLE: No UI learning required, voice for everyone
5. PRIVATE: Local-first option with zero Titan servers
6. FLEXIBLE: Cloud OR local, user chooses
7. AFFORDABLE: Free tier for solos, \$29-\$999 for teams
8. COMPLETE: Nothing else needed (no tool fragmentation)

COMPETITIVE LANDSCAPE:

Traditional approach:

- └ Dispatch system: \$100-500/month
- └ Accounting software: \$50-300/month
- └ CRM: \$50-150/month
- └ Project management: \$50-200/month
- └ Marketing/design: Hire designer (\$3-5k/month)
- └ Compliance tracking: Manual + consultant
- └ Total: \$3,500-6,650/month minimum

TITAN BOS:

- └ Everything in one: \$0-999/month
- └ Includes AI, voice, offline, privacy, compliance

MARKET OPPORTUNITY:

Service industry market size: \$1.2 TRILLION/year
Addressable market (tech adoption): \$50 BILLION
Current market penetration: <1%

If 1% of market adopts TitanBOS:

- └ Customers: 500,000+ businesses
- └ Revenue: \$5-10 billion/year
- └ Jobs created: 5,000+ (engineering, support, sales)
- └ Market transformation: Voice becomes primary UI

 TITAN BOS: THE COMPLETE VOICE-CONTROLLED BUSINESS OPERATING SYSTEM


"Every service business should be able to run their entire company by just talking to their AI assistant. That's Titan BOS."

This is Titan Studio — the creative & marketing design component of TitanBOS, powered by CreatiCore AI, generative models (DALL-E, Stable Diffusion), and voice control via Titan Omni. The entire ecosystem is now complete:

1. Titan Omni - Voice command center
2. Titan Pro - Business dashboard
3. Titan Command - Dispatch system
4. Titan Go - Field operations
5. Titan Trust - Compliance & risk
6. Titan Portal - Customer app
7. Titan Studio - Creative & marketing

Plus Titan Zero (AI intelligence) powering everything with 6-core reasoning.

This is a complete, voice-first, privacy-first, fully integrated business operating system that requires no other software — suitable for solos, startups, and enterprises up to 1000+ people. Zero vendor lock-in, complete data ownership, and enterprise-grade reliability.

Market ready: Q4 2025. 